

Nova Frontier - developer guide

This is the long version of `GAME.md`: how a 3D, landscape real-time strategy game is put together on top of the engine, written so that you could rebuild it yourself. It assumes you have read the engine guide (`docs/GUIDE.md`): the loop, the services, the layers and the rule that the engine draws nothing.

1. The game

One player against the computer on a generated 96 x 96 map. Each side starts with a **Hub** (4 x 4 cells), four **workers**, 250 minerals and 10 supply. Everything is a *cell* (the map is a grid); a worker is about 0.7 of a cell wide.

Unit	Cost	Supply	Trained at	HP	Armor	Speed	Weapon
Worker	50	1	Hub	40	0	3.0	5 dmg, melee (0.9), ground
Trooper	50	1	Barracks	55	0	3.2	7 dmg, range 5.5, ground + air
Tank	150 + 100 gas	3	Factory	170	1	2.4	30 dmg + splash 1.3, range 8, ground only
Skiff (flies)	125 + 100 gas	2	Airfield	95	0	4.6	9 dmg, range 5.5, ground + air

Building	Cost	Needs	Gives
Hub	400	-	+10 supply, receives minerals, trains workers
Depot (2 x 2)	100	-	+8 supply
Barracks (3 x 3)	150	Hub	trains Troopers
Refinery (3 x 3)	75	Hub, on a geyser	gas (1.6 a second)
Factory	150 + 100 gas	Barracks	trains Tanks
Airfield	150 + 100 gas	Factory	trains Skiffs
Turret (2 x 2)	100	Barracks	12 dmg, range 7.5, ground + air

Supply is capped at 120. A worker carries 5 minerals a trip (2.2 s to mine); a patch holds 1500. A building costs its money when the worker **starts** it (not when you click), takes the builder away from mining while it rises, and a cancelled one returns 75%. Each building queues five units. You win by destroying **every building** of the other side; units alone do not keep a side alive.

Every number above lives in `sim/config.ts`; the tables in this guide are copies, that file is the truth.

2. Determinism: the idea everything rests on

The simulation (`src/game/sim/`) is a pure function of a seed and a list of commands. It never reads a clock, never calls `Math.random` , and, going further than the architecture test demands, never uses `sin` , `cos` , `pow` or `hypot` (their results can differ in the last bit between devices; `sqrt` and the four operations are exact). It runs at **20 ticks a second** (TPS), independent of the frame rate: the engine calls `update` at a fixed 20 Hz and the view interpolates between ticks (`px` , `py` is where a unit was at the start of the tick; `render(alpha)` blends).

What this buys:

- the same seed is the same map, the same computer opponent and the same fight on every device;
- the balance tests can play thousands of games in seconds, headless;
- the view can be thrown away and rewritten without touching a rule.

Two details that are easy to get wrong:

- **Who acts first has an edge.** In a tick, a unit that kills its target stops that target firing back, so whoever is updated first wins ties. `stepMatch` therefore walks the entity list forwards on even ticks and backwards on odd ones (and alternates which computer player thinks first). Before this, normal-vs-normal went 9 to 1 for side 0; with it, 11 to 9 over twenty maps.
- **Randomness is state.** The only RNG is the engine's seeded one, held in the match (`m.rng`).

3. The map generator (`sim/map.ts`)

`generateMap(seed)` is an algorithm, not a lookup of hand-made levels:

1. **Layout.** Three base sites are fixed for the side in the west (the start and two expansions); the other side's are the same points turned half way round the centre (`x -> 95 - x` , `y -> 95 - y`). Each base grows eight mineral patches in an arc and one or two geysers; the second side's resources are the first side's, turned. So both players get *the same ground*, which is what makes the balance tests mean something.
2. **Rock.** 18 to 26 blobs (ragged edges from a hash of the cell) and 2 to 4 ridges, each with a gap in it so there are chokepoints. Every stroke is painted **and its half-turn twin**.
3. **Clear.** A disc of radius 13 around every base is cleared, resources are cleared, and a two-cell border is closed.
4. **Flip.** Half the maps are mirrored top to bottom so the start is not always the south-west.
5. **Prove.** A flood fill (8 directions, no corner cutting) from the first base must reach every other base. If it does not, the next attempt for that seed is tried (up to 12); the last resort is an open field.
`tests/map.test.ts` checks determinism, the half-turn symmetry (cell by cell, minerals included), eight patches a base, clear bases and a closed border, and connectivity for 40 seeds.

4. Moving: pathfinding and crowds (`sim/path.ts` , `sim/units.ts`)

- **A*** on the cell grid, 8 directions, never cutting a corner, with a binary heap whose buffers are reused (a `generation` stamp avoids clearing arrays between searches).

- **String pulling.** The cell route is straightened wherever a straight line of the unit's width is clear (`lineClear`), so units walk in long straight legs rather than staircases. A fat tank keeps more clearance than a worker.
- **Goals that cannot be reached** go to the nearest free cell; **blocked starts** are nudged out. A route is `complete` or ends at the nearest reachable point.
- **Buildings change the map.** `m.blocked` is the rock plus every building and resource. Placing or destroying one bumps `pathVersion`; a walking unit that notices its route is old, finds it again.
- **Stuck units.** A unit that has not moved for 24 ticks re-finds its way; after five tries it gives up.
- **Crowds.** Units are bucketed by cell each tick and pushed apart (`separate` , 0.35). A group order spreads the group over a grid of spots around the click (`formation`), the units nearest the target taking the front spots, so a crowd does not all pick the same cell.
- **Air units** ignore all of it: a straight line.

5. Economy and building (`sim/economy.ts` , `sim/construction.ts`)

A worker's `gather` order is a small state machine: *to the patch -> mine (2.2 s) -> to the nearest Hub -> drop off -> the least busy patch near the last one*. "Least busy" counts how many of that player's workers are headed for each patch, so eight workers spread over eight patches. Arriving is "within 0.5 cells of the edge" (a worker cannot stand exactly on a corner). `approachPoint` picks the point on the building's edge nearest the unit, not its centre, so workers do not all queue at one face.

`checkPlacement` is the one place that knows whether a building may go somewhere: it snaps to the grid (even sizes sit on a corner, odd sizes in a cell's middle), snaps a refinery onto a geyser, checks the requirement, rock, other buildings and the edge of the map, and (for a person, not the computer) that the ground has been **explored**. It returns a reason string the HUD shows. Units standing on the spot are moved out when the building starts.

6. Combat and vision (`sim/combat.ts` , `sim/vision.ts`)

- **Who can hit whom** is two flags on a weapon (`air` , `ground`). A tank cannot shoot a skiff; the card still lets you try and says "Cannot attack that"; a mixed group sent at a skiff has the troopers attack and the tanks walk up.
- **Damage** is `max(1, damage - armor)` . Splash (the tank) hurts ground enemies near the target for half; it never hurts your own side or flyers.
- **Target choice** scores things that shoot back first, then other units, then buildings, then distance. Idle units look for enemies within weapon range + 3.5; **hold** only shoots what is in range; **attack-move** fights what it meets, then goes back to what it was doing (`resume`).
- **Fog.** Each player has a grid: 0 never seen, 1 seen before, 2 in sight. It is redrawn every 4 ticks from the sight radius of every entity you own. You always see your own things; enemy buildings and resources once the ground has been seen; enemy **units only while in sight**. Weapons cannot target what is not visible (`canSee`).

7. The computer player (`sim/ai.ts`)

It plays by the same rules through the same commands (`cmdBuild` , `cmdTrain` , `cmdMove`): it mines, builds, trains and attacks, and never gets a free unit. Every `think` ticks it counts what it has and walks down a priority list: keep workers training up to a target; add depots before supply runs out; a refinery; a barracks, factory and airfield up to the level's targets; train from whatever is free, choosing the most expensive unit it can afford and *saving* for gas units rather than letting cheap ones eat the minerals; expand when rich; send the army out as a **wave** when it reaches a supply target (which grows each time); defend the base when something threatens it.

	Easy	Normal	Hard
Thinks every	2.5 s	1.5 s	1.0 s
Worker target	12	18	24
First wave / growth (supply)	14 / +6	20 / +8	24 / +10
Mining trips pay	80%	100%	125%
Barracks / factories / airfields	2 / 1 / 0	3 / 1 / 1	4 / 2 / 2

Measured by `tests/balance-*.test.ts` and by hand (20 normal-vs-normal maps went 11 to 9; hard beat easy 12 of 12 and beat normal 15 of 16; a game lasts 8 to 15 minutes; a tick costs 0.15 to 0.25 ms).

8. Drawing (`src/game/view/`)

The 3D canvas is a Three.js `WebGLRenderer` created by `createWorld3d` and **inserted under the engine's transparent Pixi canvas** (`view.host` , `boot(..., { transparent: true })`). Its CSS box is set every frame to the letterboxed rectangle of the 640 x 360 world, so everything else (the 2D overlay, the DOM stage, the controls) shares one coordinate space: *engine world coordinates*.

- **Models** (`models.ts`) are lists of simple solids (box, cylinder, cone, sphere, octahedron, dome) with a position, size, colour (fixed, or the team colour) and optional flags (`glow` : unshaded; `spin` : turns with the unit; `show` : only when something is true, like a worker carrying minerals). There are no model files and no textures.
- **Instancing**. Each part of each model is one `InstancedMesh` . A frame walks the entities once, and for every visible one writes a matrix and a colour into the instance buffers: about 90 draw calls for the whole game however many units there are. Hurt entities flash white, buildings going up are scaled in height and dimmed, units face the sim's facing at a limited turn rate, and skiffs hover.
- **Fog is in the material** (`materials.ts`). A 96 x 96 texture holds how bright each cell is (0 / 0.46 / 1), blurred a little and faded over time. Every material gets a small `onBeforeCompile` patch that computes the world position of each fragment and multiplies by that texture. So rock, buildings, units and ground darken together, with no overlay plane to line up with the tilted camera.
- **Camera** (`world3d.ts`): a perspective camera at a fixed 56 degree pitch looking at a ground point `cam.x` , `cam.y` from distance `cam.zoom` ; `setViewOffset` shifts the picture up so the camera target sits above the bottom panel. Picking casts the pointer ray at several heights through each entity's body and chooses units over buildings over resources; a box select projects entity centres to the screen.

- **Terrain** (`terrain.ts`): a canvas ground texture made from the seed (tile tones, soft patches, paler pads under bases, grid lines every eight cells) and the rock as one instanced block per cell with a hashed height, turn and tone. **Effects** (`effects.ts`): three instanced pools (tracers and shells, sparks, rings on the ground), additive and unlit.
- **The 2D overlay** (`overlay.ts` , `Pixi`): health and progress bars and the selection rectangle. **The minimap** (`minimap.ts`) is a plain 2D canvas redrawn 30 times a second.

The view reads the match and never writes it. It has a dev-only reading of cost: about 1.2 ms of CPU a frame in a 115-entity battle.

9. Controls and the HUD

- `session.ts` is *what the player is doing*: the selection, the mode of the next click (`move` , `attackMove` , `rally` , `place`), the build submenu, control groups, and the **command card**. Functions like `orderAt(target, x, y)` decide what a right click means (attack, mine, help build, rally, walk); `cardFor` returns the nine buttons with their costs and, when disabled, the reason. It has no DOM, so `tests/session.test.ts` drives it directly.
- `controls.ts` turns pointer and key events into calls on the session. **Mouse**: left click selects (drag a box; double click selects every unit of that kind on screen), right click (or Ctrl + click, for a trackpad) orders, middle drag pans, the wheel zooms about the cursor. **Touch**: tap selects or orders by what is under the finger, drag pans, **press-and-hold then drag** boxes, two fingers pinch and pan, a long press on the minimap orders, and an x on the selection panel lets go of the selection (a tap on the ground would send the units there). There is no right click on touch: a tap on an enemy attacks, on minerals mines, on the ground moves. Building by touch is *choose, tap where, press Build here*, because there is no hover.
- `ui/` is the DOM: top bar, minimap box, selection panel, the 3 x 3 card, and the menus. It takes plain values (`HudInfo` , `SelectionInfo` , `CardButton[]`) and reports presses through callbacks; it never sees the match. The HUD root is `pointer-events: none` , only its buttons take presses, and presses on the world reach the controls because the DOM stage is a *sibling* of the game host.
- A phone held upright shows a "turn your phone sideways" screen, and the game pauses itself when the page is hidden or turned upright.

Phones first

The target is a phone held sideways, so the design starts from the thumb.

- **Layout.** `createUi` adds a `touch` class to the stage when the primary pointer is coarse (`(pointer: coarse)`), and `ui/styles.css` ends with a `.nova.touch` block: buttons at least 44 px (`min-height: 44px` , chips 42 px), the 3 x 3 command card absolutely positioned so it can be taller than the bottom panel (its empty slots are invisible and take no presses, so the card never blocks the battlefield), keyboard hints hidden, text a notch bigger. Sizes are in world units (`calc(var(--u) * n)`), which are about 1.0 to 1.2 px on a phone, so the floors in pixels are what really guarantee the size.
- `e2e` **"phones"**. The HUD is measured at 667 x 375, 800 x 360, 844 x 390, 915 x 412 and 932 x 430: every button at least 42 px, nothing off screen, the three panels not overlapping, the battlefield keeping the top 60%; and the menu fits.

- **Fullscreen, landscape lock, wake lock** (`index.ts`). On a touch device the Battle press (a user gesture, which browsers require) calls `requestFullscreen` and `screen.orientation.lock('landscape')`, and a screen wake lock is held during a battle (released at the menu and the result, taken again when the page returns). All of it is wrapped in try/catch: Safari on an iPhone supports none of the first two, and the game plays the same. The Settings screen has a "Fullscreen on phones" switch.
- **Adaptive resolution** (`quality.ts`). A phone's GPU cannot be known, so the game measures: a smoothed frame time above about 32 ms lowers the 3D resolution one step (x 0.85, floor 0.6), at most every 3 seconds, and never raises it again (so it cannot flap). One long hitch does not count. `tests/quality.test.ts` covers it; `?quality=N` pins it in dev so screenshots and tests stay sharp.
- **Installable.** `public/manifest.webmanifest` (fullscreen, landscape), PNG icons made from the favicon's artwork (including a maskable one for Android and an apple-touch-icon), and the Apple meta tags in `index.html`: "Add to Home Screen" gives a fullscreen landscape app, the only way to get fullscreen on an iPhone.

10. The flow (`index.ts`)

Phases: **menu** (a real computer-against-computer battle plays behind it, fast-forwarded two minutes so it opens busy, the camera following the fighting), **playing, paused, result**. Starting a battle creates a match and a session, loads the map into the 3D world, gives the Hub a rally point on the minerals (so new workers mine) and selects it. Sim events (`shot`, `death`, `built`, `alert` ...) are turned into effects, sounds, toasts and minimap pings; only the player's own match makes noise. At speed above 1 the flow interpolates across the whole batch of ticks so the picture stays smooth.

Ads: the **Supply drop** (rewarded, once per battle) and an interstitial between battles; both go through the engine's guarded ad service, so a failed ad is just "no reward".

11. Tests, and what is not verified

`npm run check` runs the type check and Vitest: the sim (map, pathfinding, economy, building, combat, vision, commands), the computer player (determinism, rule-keeping over a whole match, fairness, the difficulty ordering), `session.ts`, the profile, and the architecture rules. `npm run e2e` drives the real game in Chrome with software WebGL: menu, mining, training, placing a building with the mouse and by touch, a fight, victory and defeat, the minimap, pause, groups, the full tech tree. Open the screenshots in `e2e/screenshots/`: a passing test does not prove the picture is right.

Not verified: a real phone. The touch controls were exercised with Chrome's emulated touch events, which are faithful for pointer logic but not for finger size, palm contact or latency; the frame rate was only measured as CPU time (software WebGL cannot say how a phone GPU will do, which is why the resolution adapts by itself); haptics and sound were not heard; fullscreen, the landscape lock and the wake lock were not tried on a device; installing the page to a home screen was not tried. The native Android project is generated (`npm run android:setup`) but was not run on a device, and iOS needs a Mac.

12. How to change things

- **Balance:** edit `sim/config.ts`, then run `npm test`: the balance tests will tell you if the computer levels stopped being ordered or fair.
- **A new unit:** add it to `UNITS` (and `UNIT_TYPES`), a model in `view/models.ts`, an icon name in `ui/icons.ts` if you want a new glyph, and the building that makes it lists it in `produces`. The computer will use it if you add it to its unit choice in `ai.ts`.
- **A new building:** `BUILDINGS`, a model, and a line in `BUILD_MENU` (`session.ts`).
- **A new map shape:** `build` in `map.ts`; keep every stroke symmetric (paint the twin) and keep the connectivity proof.
- **Look:** palette in `view/palette.ts`, light and camera at the top of `view/world3d.ts`, the HUD in `ui/styles.css` (sizes are in world units: `calc(var(--u) * 12)`).

13. Ship

`npm run android:setup` builds and creates the Android project (the setup script locks it to landscape, from `"orientation": "landscape"` in `capacitor.config.json`); `npm run android` runs it (JDK 21 and the Android SDK are needed). The GitHub Pages workflow on `main` publishes this branch as a playable page and its guide as a PDF. Test ad units are used until you set `VITE_ADS_TESTING=false` and the real ad unit ids (see the engine guide).